

# Lecture 6: Leases and LBFS (Reducing Communication)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau  
University of Wisconsin — Madison  
Spring 2026

1

## Today's Topic: Leases and LBFS

**Leases** - Cary G. Gray and David R. Cheriton, [Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency](#), Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP), December 1989, **SIGOPS Hall of Fame Award**

**LBFS** Athicha Muthitacharoen, Benjie Chen (MIT), David Mazieres (NYU), [A Low-Bandwidth Network File System](#), SOSP'01

Why Today's Papers?

- Leases:
  - Useful for delivering cache consistency in presence of host and communication failures
  - Simple mechanism: Assumes physical clocks
  - Used in many systems
- LBFS:
  - Good summary of related work (NFS, AFS, Coda, Leases)
  - Neat techniques (used in storage deduplication too): Hashes and fingerprints
  - Reasonable design decisions building on top of NFS/AFS

2

## Motivation for Leases

Node thinks it has “authority” for some action, but something fails

- Network partition or crash

Authority:

- Cache: serve, or modify data
- Control metadata: directory contents, placement, ownership
- Coordination: leader of consensus algorithm

Problem:

Authority easy when all nodes alive + connected, difficult under failure

Hard to continue progress and remain available

3

## Goal: Time-Bounded Authority

| Mechanism     | Authority              | Failure behavior         |
|---------------|------------------------|--------------------------|
| Locks         | Explicit, revocable    | Can block forever        |
| Callbacks     | Explicit, revocable    | Requires connectivity    |
| <b>Leases</b> | Implicit, time-limited | Authority expires safely |

Other approaches for authority don’t behave well under failure

**Lease is promise that *time* will clean up after failures**

4

## Lease: Consistent Caching on Clients

### Lease:

- Contract that gives its holder specified rights for a limited period of time

### Term:

- Time period holder has control over writes;  
Server must get approval from holder for writes during term

### Steps for client to read into local cache:

- Client fetches data from server and obtains lease for some term
- Future reads can remain local during term (guaranteed to have up-to-date consistent copy)
- Client asked for approval of any writes on server during term

### Client writes back to server:

- If no clients have lease, server can write
- Otherwise, server gets approval from all lease holders (or waits until leases expire)

5

## Leases: Renewals

### Lease renewal:

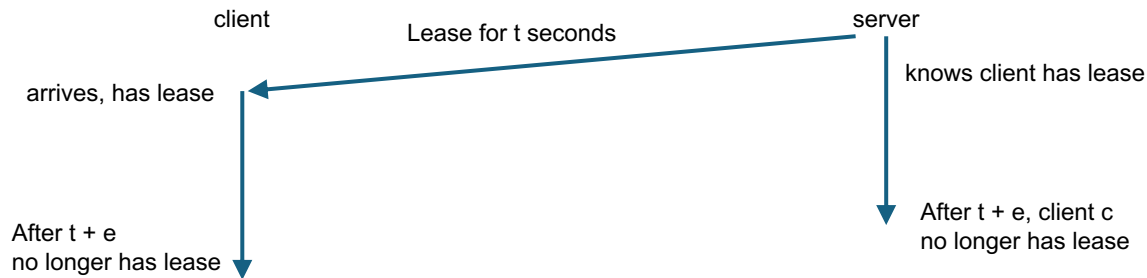
- Client asks server to extend lease before it expires

### If lease will soon expire, should client request extension or not?

- Yes: Improves response time for local reads
- No: Increases load on server
- Worst case: idle client requests extensions when file not accessed

6

## Leases: Depend on Physical Clocks

Term  $t$ 

### Implications

1. Do not need to be perfectly synchronized (i.e., don't agree on physical time)
2. Clocks should progress at approximately same rate
3. Client can be faster and still correct
4. Communication time ( $e$ )  $\ll$  Lease term  $t$

7

## Lease: Term Length can be Adaptive

### Advantages of Short Lease Terms

1. Minimize delay after failures
  - Client  $c_1$  partition: Must wait term before giving to new client  $c_2$
  - Server crash: Must wait term to ensure no client is using
2. Minimize **false sharing**: treat as if conflict when none exists
  - Can't get lease because held by client that doesn't need it anymore
3. Reduce server storage requirements
  - Server must track all granted leases
  - Forget about leases when expire
  - Extreme: no leases  $\rightarrow$  no state!

### Advantages of Long Lease Terms

- **Performance**: More efficient if client accesses data repeatedly
- Don't need to ask for extensions as frequently
- Best when: Little sharing

8

## Leases: How to set Term?

### Heavily Write-shared file:

- Lease term = 0
- Why?
  - Writes happen immediately, don't have to wait for client or term expiration
  - Implication: Can't use or trust cached file (true)

### Distant client:

- Lease term = Longer
- Why?
  - Account for propagation delay and time asking for extension

9

## Leases: Two Extremes of AFS

### Version 1: Poll on every file open

Lease term = 0

Problem? Extra overhead/checking when no sharing

### Version 2: Callbacks on server – notify client of file changes

Lease term = infinite

#### Problem?

If communication with client fails, allows updates to proceed on server, while client uses old data (causes inconsistency)

#### Alternative?

Short-term lease, client would contact server when expires

10

## Leases: Optimizations for “Installed” Files

**File Properties:** Widely shared, heavily read, infrequently written

Use small number of leases across multiple files

Server multi-casts extension to all clients periodically; **why good idea?**

- Clients don't have to request extensions

**What to do when file is written?**

- Don't extend lease in multi-cast

**When can write proceed?**

- When lease expires (same as usual)
- Is this longer latency than asking clients for approval?
  - Probably not, since probably one would be unreachable

**Advantage for server state?**

- Doesn't need to track individual clients

11

## Leases: Summary

**Basic idea**

- Authority over resource while have lease; must be asked to accept changes while hold lease
- Passing of time automatically handles failures

**Trade-offs:** Determining best lease term

- **Too long**
  - False sharing
  - Must wait longer after failure
- **Too short**
  - Not safe if doesn't account for network latency
  - Overhead of continued renewals
- Adaptive: Set lease term depending on workload, environment

**Advantages**

- Handles host and communication failures
- Scales well, easy fast path
- Simple mechanism + assumptions
  - Just physical clocks, network latency  $\ll$  term

12

# Low-Bandwidth Network File System (LBFS) - 2001

## Motivation?

- Client/server FS for low-bandwidth networks

## Goal: Metric?

- Minimize amount of network traffic (bandwidth needed);
- Not network latency, not round-trips
- **Related technique?**
  - Compression

## Workload Assumptions?

- Small changes to files are common
- Many similar versions of files exist
- Examples: auto-save, compiling object files, version control

13

# Related Work: Reduce Network Traffic?

## AFS:

- Callbacks – don't refetch file if not changed

## Leases: Lease terms

- Reduces state on server
- Helps with fault tolerance

## NFSv4:

- Delegation with leases
- COMPOUND RPC

## Coda:

- Works in extreme of NO bandwidth
- Log updates, send modifications on reconnection
- Don't send deleted files

14

## LBFS: Basic Idea

Don't send data chunks over network if client/server already has it

### How to identify unique chunks?

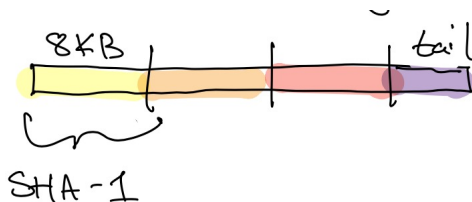
- Use SHA-1 hash (no collisions)

### Trade-off in identifying identical chunks?

- Examine more parts of files:  
Higher overhead of calculating and tracking hashes
- Examine fewer parts of files:  
Less likely to find identical chunks

15

## Option 1: Aligned, non-overlapping chunks



**What will it find?** aligned identical chunks across any set of files

**What won't it find?** anything unaligned

#### Pros:

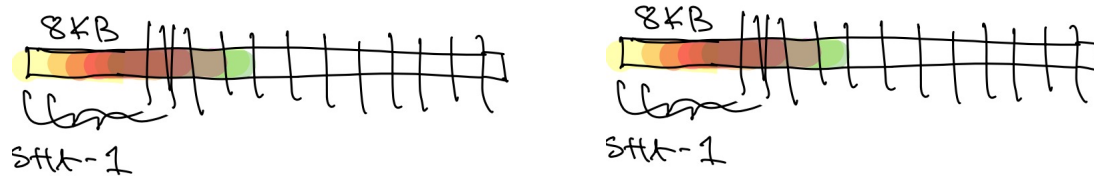
Easy, few hashes, low overhead for storage and computation

#### Cons:

Will not find many existing matches

16

## Option 2: Sliding-Window of Overlapping Chunks



### Pros:

Easy to find matches at any alignment

### Cons:

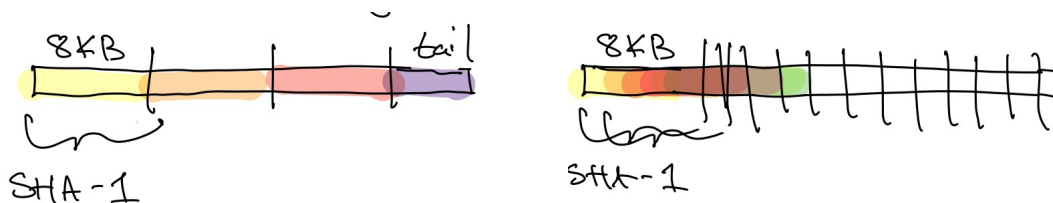
Overhead for storing / tracking many hashes

One change impacts many overlapping chunks and hashes → many recalculations

17

## Option 3: Related Work – rsync

Combination

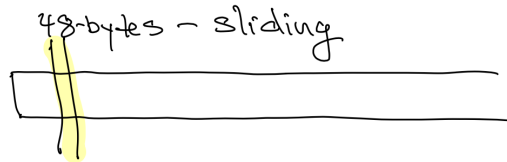


### Limitations?

- Made assumptions about filenames
- Sliding window really expensive

18

## LBFS Chunk? Not Fixed Size



Compute Rabin's fingerprint over 48-bytes

$$R = \underbrace{\hspace{2cm}}_{13 \text{ bits}} \Rightarrow \text{Breakpoint?} \quad (0x32a)$$

**Probable size of each chunk?**

probability of matching breakpoint?  $2^{13}$  likely size of chunk?  $2^{13}$

**Represent each chunk?**

SHA-1 hash

19

## LBFS: Properties w/ Rabin's Fingerprint?



**Common cases: Which chunks change?**

- Overwrite data in c4? just c4
- Add data to c4? just c4
- Delete data from c4? just c4

**Less common**

- Add data that matches fingerprint in c4?
  - Two new (smaller) chunks c4a and c4b
- Delete data was fingerprint at end of c4?
  - Merge old c4 and old c5 into one new (larger) chunk

**Practical Issues**

- Too small or too large chunks (all zeros) → define min and max chunksize
- Suppress breakpoint if  $< \text{min}$ ; force breakpoint if  $> \text{max}$

20

## LBFS: Chunk Database on Client & Server

Map 64-bit key (from 160 bit SHA-1 hash) <-> (file, offset, count)

Update on all file changes

### What if file modified outside of LBFS?

DB says: K1 -> file A

But file A now file A'

### Implication?

Must recheck that file A' matches K1

Treat only as a hint!

21

## LBFS Cache Consistency?

AFS with Leases

Avoids unnecessary traffic w/ server

### Why whole file?

Easier to do chunk comparisons when open/close

Read/write local and simple

### What to do on open?

If still have lease, use local copy

If lost lease, check if attributes have changed; if not, use local copy

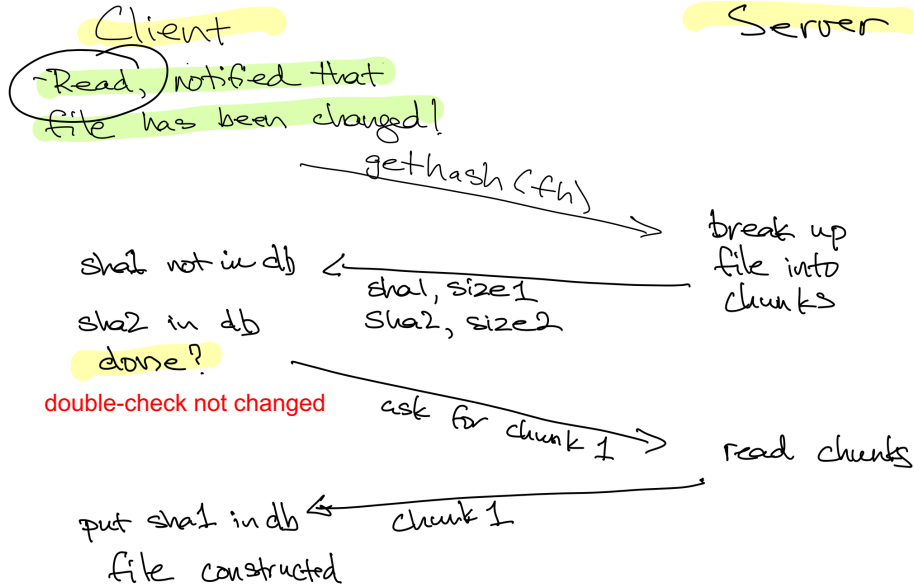
If file is different, just fetch what changed!

### What to do on close?

When close file, just write back to server what changed

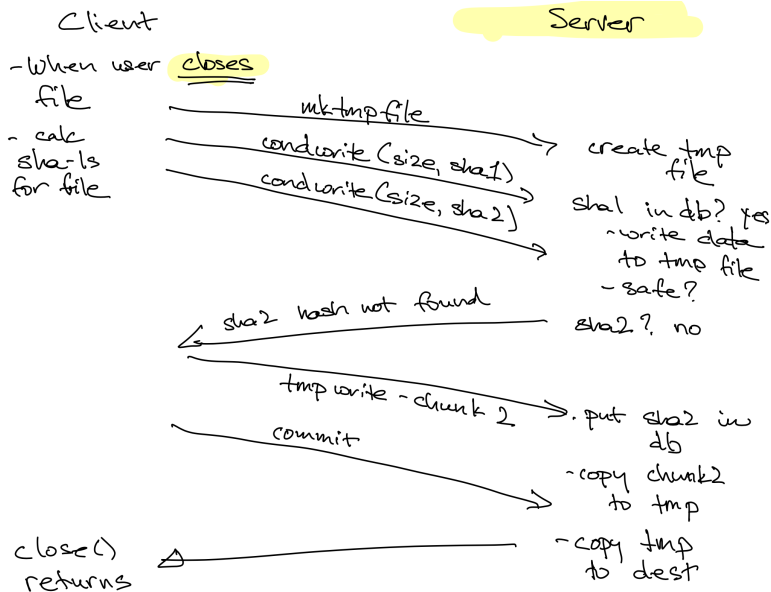
22

## LBFS: Open/Read Protocol



23

## LBFS: Close/Write Protocol



Ever worse than without?  
yes, if file completely changed

extra overhead on server

more latency for small files

What does server do at end?

cp from tmp to dest to be atomic

Problem with  
Unix filesystem  
semantics?

Must do cp instead of rename  
since dest must keep old inode  
number

24

## Summary of LBFS

Minimize network traffic

LBFS: Only sends data other side doesn't already have

- Assumes small changes to files
- Use SHA-1 hash to identify/name chunks
- Rabin fingerprints: Break data up into variable-sized chunks; data changes impact minimal chunks
- Chunk database: Only hints! Can't be trusted since files can be modified outside of LBFS

25

## Announcements

Reading Group Week 3: New groups, Due Sunday evening

Mini-Project 1: RPC for Client/Server KV Store – Due this Sunday

- Signup for 10-minute demo w/ TA – Monday, Tuesday, Wednesday

Mini-Project 2: Available early next week

- Extend Mini-Project 1:  
Add durability and key-space partitioning ACROSS independent servers
- Can work w/ same project partner or different one

Friday:

- **Ordering -- Logical Clocks and Distributed Snapshots**

26